

Order-Picking in a Rectangular Warehouse: A Solvable Case of the Traveling Salesman Problem
Author(s): H. Donald Ratliff and Arnon S. Rosenthal
Source: *Operations Research*, Vol. 31, No. 3 (May - Jun., 1983), pp. 507-521
Published by: INFORMS
Stable URL: http://www.jstor.org/stable/170620
Accessed: 24/09/2014 09:31

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at
http://www.jstor.org/page/info/about/policies/terms.jsp

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Operations Research.*

http://www.jstor.org

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

Order-Picking in a Rectangular Warehouse: A Solvable Case of the Traveling Salesman Problem

H. DONALD RATLIFF
Georgia Institute of Technology, Atlanta, Georgia

ARNON S. ROSENTHAL
Sperry Research, Sudbury, Massachusetts

(Received November 1981; accepted October 1982)

This paper addresses the problem of order-picking in a rectangular warehouse that contains crossovers only at the ends of aisles. An algorithm is presented for picking an order in minimum time. The computational effort required is linear in the number of aisles. The procedure has been implemented on a microcomputer. A 50-aisle problem requires only about 1 minute to solve.

ONE OF the most fundamental item retrieval problems associated with warehousing and materials handling is what we call the simple "order-picking problem." An order consists of a subset of the items stored in a warehouse. When receiving an order, the warehouse dispatches a vehicle from the shipping area to collect or "pick" the items in the order and transport them back to the shipping area. The objective is to minimize distance or time traveled by the vehicle. We assume that a vehicle picks only one order at a time and that an order does not exceed the vehicle's capacity.

For an arbitrary aisle configuration within a warehouse, the order-picking problem is easily recognized as a variant of the well-known and difficult to solve traveling salesman problem (e.g., see Christofides [1975]). Fortunately, the most common warehouse aisle configuration is that given in Figure 1. For this configuration we will present an algorithm for optimally solving the order-picking problem which is linear in the number of aisles. The algorithm is hence fast enough to be applied in any realistic size warehouse. The memory requirement is small enough to allow its implementation on a small computer. An experimental code has been developed in Basic for an Apple III microcomputer. A 50-aisle problem requires about 1 minute to solve. The number of items in the order has little effect on the solution time using this algorithm.

Subject classification: 491 a solvable traveling salesman problem, 587 warehouse order-picking.

Operations Research
Vol. 31, No. 3, May–June 1983

0030-364X/83/3103-0507 $01.25
© 1983 Operations Research Society of America

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

Two different special cases of the traveling salesman problem for which efficient solution methods have been developed are given by Gilmore and Gomory [1964] and by Lawler [1971].

1. GRAPH REPRESENTATION

Consider an order containing m items to be picked in a warehouse with

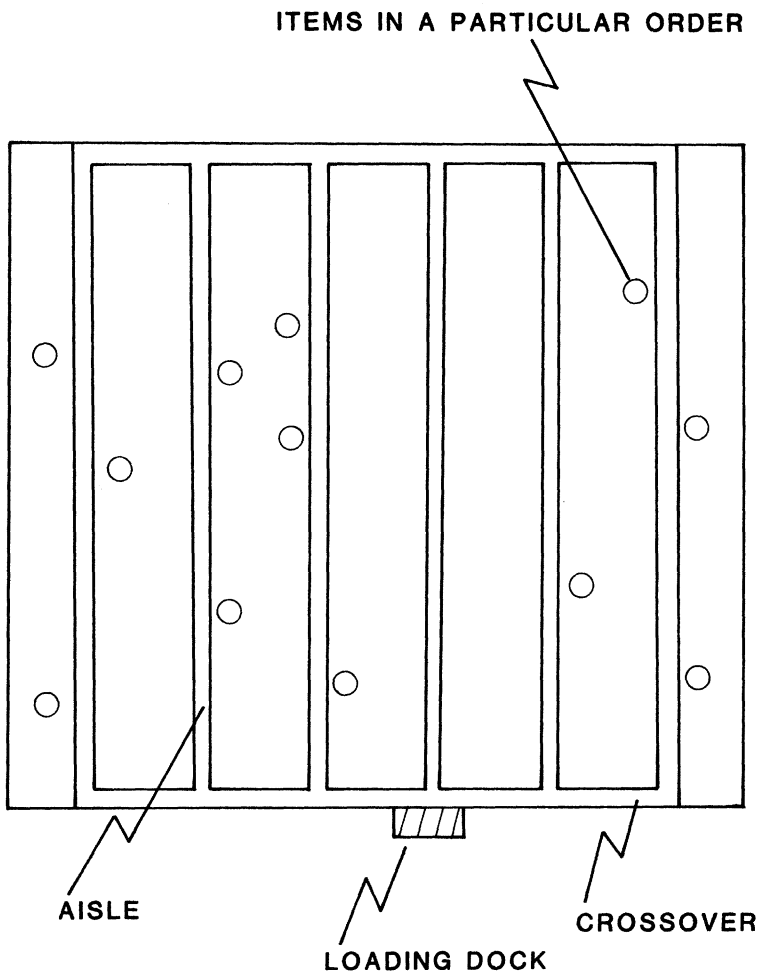


Figure 1. Warehouse aisle configuration.

the aisle configuration illustrated in Figure 1. Define a graph G by associating a vertex v_0 with the shipping area location, a vertex v_i with the location of each item $i = 1, 2, \cdots, m$ in the order, and vertices a_j and b_j with the ends of each aisle $j = 1, 2, \cdots, n$. (Note that aisles are numbered from left to right.) Connect any two vertices in G that corre-

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

spond to adjacent locations in the warehouse by an unlimited number of parallel arcs, each with length equal to the direct distance between the two locations. The graph in Figure 2 corresponds to the warehouse and order depicted in Figure 1. For simplicity, the figure represents the parallel arcs connecting adjacent vertices by a single arc. Also for simplicity, we have depicted v_0 as being between vertices a_4 and b_4 instead of coinciding with b_4.

An order picking tour is a cycle in G that includes each of the vertices v_i for $i = 0, 1, \cdots, m$ at least once. Note that since each adjacent pair of

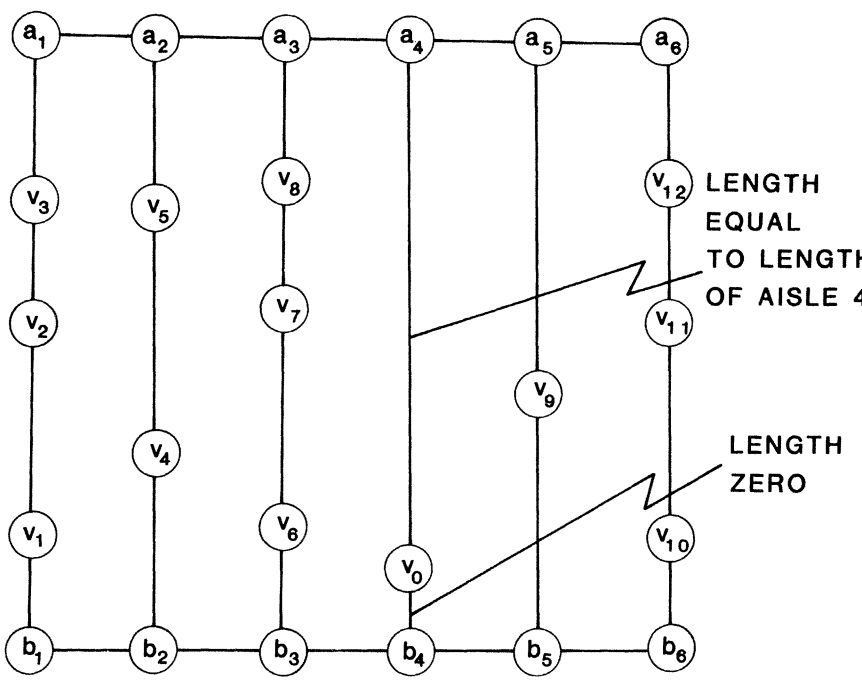


Figure 2. Order-picking graph where each arc represents an unlimited number of parallel arcs.

vertices is connected by an unlimited number of parallel arcs, we can assume, without loss of generality, that an order-picking tour contains any arc at most once. The order-picking problem is then to find a minimum length order-picking tour in G.

A subgraph T of G (we will denote this as $T \subset G$) that contains all vertices v_i for $i = 0, 1, \cdots, m$ will be called a tour subgraph if there is an order-picking tour that uses each arc in T exactly once. The following characterization of a tour subgraph is a specialization of a well-known theorem on Eulerian graphs (e.g., see Christofides).

THEOREM 1. *A subgraph $T \subset G$ is a tour subgraph if and only if* (*a*) *all*

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

vertices v_i for $i = 0, 1, 2, \cdots, m$ have positive degree in T; (b) excluding vertices with zero degree, T is connected; and (c) every vertex in T has even degree or zero degree.

Given a tour subgraph, we will show in Section 4 that an order-picking tour can be very efficiently determined. Hence, by developing an efficient procedure for finding a minimum length tour subgraph, we can efficiently solve the order-picking problem. The following corollaries of Theorem 1 are useful in characterizing a tour subgraph.

COROLLARY 1.1. *A minimum length tour subgraph contains no more than two arcs between any pair of vertices.*

COROLLARY 1.2. *If $(P, \bar{P})$ is any node partition of a tour subgraph, there is an even number of arcs with one end in P and the other end in $\bar{P}$.*

For any subgraph $L \subset G$, a subgraph $T_j \subset L$ is a L *partial tour subgraph* (PTS) if there exists a subgraph $C_j \subset G - L$ ($G - L$ denotes the graph that remains after all arcs and vertices in L have been deleted from G) such that $T_j \cup C_j$ is a tour subgraph of G. The subgraph C_j will be called a *completion* of T_j.

Let L_j^- be the subgraph of G consisting of vertices a_j and b_j together with everything in G to the left of a_j and b_j. Let A_j be the subgraph of G consisting of vertices a_j and b_j together with everything in G between a_j and b_j. Finally, let $L_j^+ = L_j^- \cup A_j$. Figure 3 illustrates these definitions. We use L_j to indicate that a result holds if we let $L_j = L_j^-$ or $L_j = L_j^+$.

THEOREM 2. *Necessary and sufficient conditions for $T_j \subset L_j$ to be an L_j PTS are*

(i) *for all $v_i \in L_j$, the degree of v_i is positive in T_j;*
(ii) *every vertex in T_j, except possibly for a_j and b_j, has even degree or zero degree; and*
(iii) *excluding vertices with zero degree, T_j has either no connected component, a single connected component containing at least one of a_j and b_j or two connected components with a_j in one component and b_j in the other.*

Proof. Note that the sum of the degrees of all vertices in T_j is even. Hence when condition (ii) is satisfied, if one of a_j or b_j has odd degree then both have odd degree. To prove sufficiency, we first consider the case where both a_j and b_j have odd degree in T_j. Let C_j be the subgraph consisting of all vertices in $G - L_j$, exactly two arcs between each pair of vertices in A_k for $k = j, j + 1, \cdots, n - 1$ (for $L_j = L_j^+$, $k = j + 1, j + 2, \cdots, n - 1$), one arc between each pair of vertices in A_n, and one arc from a_k to a_{k+1} and b_k to b_{k+1} for $k = j, j + 1, \cdots, n - 1$. $T_j \cup C_j$ then satisfies the condition of Theorem 1.

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

Now consider the case where both a_j and b_j have even degree, both have zero degree (i.e., there are no v_i in L_j) in T_j, or one has even degree and the other has zero degree. Let C_j be the subgraph containing all vertices in $G - L_j$ and exactly two arcs between each pair of these vertices. Again $T_j \cup C_j$ satisfies the condition of Theorem 1.

To show necessity, suppose that for some tour subgraph T, we let $T_j = T - T \cap (G - L_j)$. Note that, except for possibly a_j and b_j, no arcs in $T \cap (G - L_j)$ are incident to vertices in T_j. Therefore since conditions (i) and (ii) must hold in T, they also hold in T_j. Excluding vertices with zero

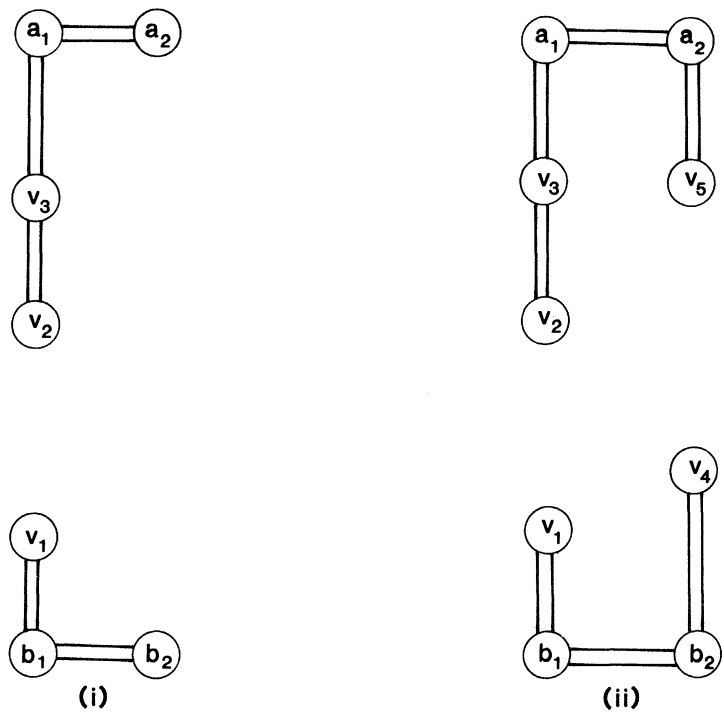


Figure 3. An illustration of (i) an L_2 PTS, and (ii) an L_2 PTSs for the graph in Figure 2.

degree, T is connected. Hence, T_j cannot have a connected component which does not contain either a_j or b_j.

We will define two L_j PTSs T_j^1 and T_j^2 to be equivalent if for any $C_j \subset G - L_j$ such that $T_j^1 \cup C_j$ is a tour subgraph, $T_j^2 \cup C_j$ is also a tour subgraph (i.e., any completion of one PTS is a completion of the other).

THEOREM 3. *Two L_j PTSs T_j^1 and T_j^2 are equivalent if*

(i) *a_j has the same degree parity (i.e., even, odd, or zero) in both and b_j has the same degree parity in both, and*

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

(ii) *excluding vertices with zero degree, both* T_j^1 *and* T_j^2 *have no connected component, both have a single connected component containing at least one of* a_j *and* b_j *or both have two connected components with* a_j *in one component and* b_j *in the other.*

Proof. Consider any $C_j \subset G - L_j$ such that $T_j^1 \cup C_j$ is a tour subgraph. C_j must contain all $v_i \in G - L_j$ and except possibly for a_j and b_j, each vertex in C_j must have even degree or zero degree. Hence, $T_j^2 \cup C_j$ contains all $v_j \in G$. Since a_j and b_j have the same degree parity in both T_j^1 and T_j^2 and all other vertices in T_j^2 are even or zero, all vertices in $T_j^2 \cup C_j$ with positive degree, must have even degree.

Now note that shrinking a connected component of a graph to a single vertex does not affect the connectivity of the graph. If in $T_j^1 \cup C_j$ and $T_j^2 \cup C_j$ we shrink the connected components of T_j^1 and T_j^2 to single vertices, the resulting graphs are identical (i.e., either T_j^1 and T_j^2 each shrink to a single vertex connected to a_j and b_j in the same fashion or they each shrink to two vertices connected to a_j and b_j in the same fashion). Therefore since $T_j^1 \cup C_j$ is connected, $T_j^2 \cup C_j$ is also connected. It then follows from Theorem 1 that T_j^2 is a tour subgraph.

We can denote the equivalence class of any L_j PTS by (degree parity of a_j, degree parity of b_j, connectivity). For the example L_2^+ PTSs in Figure 4, the equivalence classes for (i), (ii), and (iii) are respectively (zero, even, 1 component), (even, even, 2 components), and (odd, odd, 1 component). For simplicity, we will denote these equivalence classes as (0, E, 1C), (E, E, 2C), and (U, U, 1C). Note that to avoid confusion with zero, we denote odd degree parity by the letter U for "uneven."

COROLLARY 3.1. *(U, U, 1C), (0, E, 2C), (E, 0, 2C), (E, E, 1C), (E, E, 2C), (0, 0, 0C) and (0, 0, 1C) are the only seven equivalence classes for a* L_j *PTS.*

Note the (0, 0, 0C) is possible only if none of the aisles in L_j contain an item to be picked and (0, 0, 1C) is possible only if none of the aisles in $G - L_j$ contain an item to be picked (i.e., the single connected component is to the left of a_j or b_j and does not contain either of them). When all aisles are the same length and all crossovers are the same length, it is possible to show that such aisles, and hence these classes, can be eliminated.

COROLLARY 3.2. *If* T_j^i *is the minimum length* L_j *PTS for equivalence class* i *and* C_j^i *is the minimum length completion of* T_j^i*, then the minimum length tour subgraph of G is the minimum length PTS* $T_j^i U C_j^i$ *for* $i \in \{1, 2, \cdots, 7\}$.

Note that in L_n^+ the minimum length completion for the PTSs in (E, 0, 1C), (0, E, 1C), (E, E, 1C), and (0, 0, 1C) is a null graph (i.e., the

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

PTSs are already tour subgraphs). Also, since $G - L_j^+ = \emptyset$, there are no completions for the remaining equivalence classes. Hence, if we find minimum length PTSs in (E, 0, 1C), (0, E, 1C), (E, E, 1C), and (0, 0, 1C) of L_n^+, the shortest of these will be an optimum order-picking tour subgraph.

2. CONSTRUCTING MINIMUM LENGTH TOUR SUBGRAPHS

We can find a minimum length tour subgraph by considering each aisle $j = 1, 2, \cdots, n$ in sequence. Using a minimum length L_j^- PTS in each

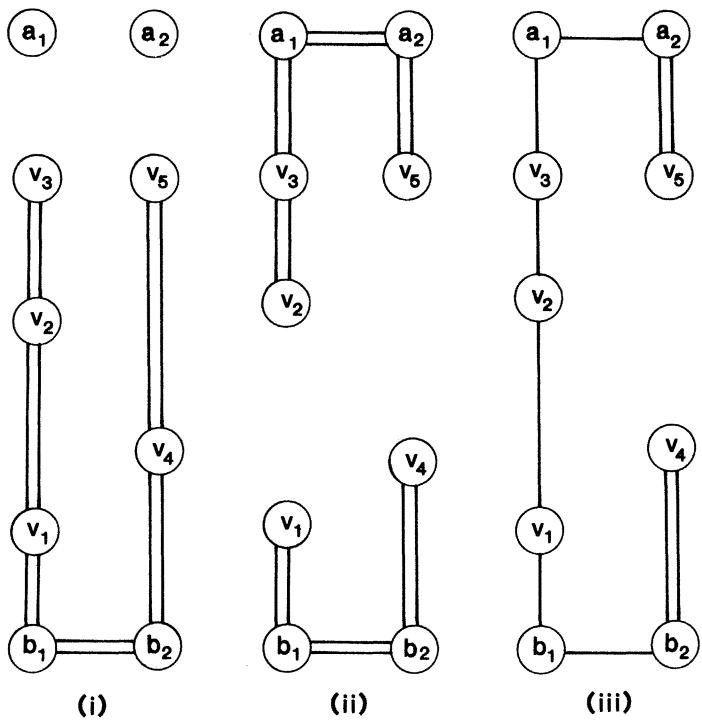


Figure 4. Examples of L_2^+ PTSs for the graph in Figure 2.

equivalence class, we construct a minimum length L_j^+ PTS. Using this L_j^+ PTSs we construct a minimum L_{j+1}^- PTS for each equivalence class. Finally, after determining the minimum length L_n^+ PTS for each equivalence class, the minimum length tour subgraph is the minimum length PTS in (E, 0, 1C), (0, E, 1C), (E, E, 1C), and (0, 0, 1C) of L_n^+.

Consider any aisle j. From Corollary 1.1, we never need more than two arcs between any pair of vertices. Hence, for a minimum length tour subgraph, the arcs corresponding to aisle j will be configured as one of the cases in Figure 5.

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

If all of the aisle lengths are the same and all crossover lengths are the same, it is possible to show that cases (v) and (vi) are not necessary since aisles containing no items can be ignored. Note that case (vi), not entering the aisle, is possible only if there is no item to be picked on the aisle.

Case (iv) is the only configuration that may not be unique. We need consider only the best (i.e., minimum length) such configuration. This is easily found by putting the gap between the two adjacent v_i's in aisle j that are farthest apart.

Table I indicates the equivalence class for L_j^+ if the minimum length configurations in Figure 5 are added to the minimum length PTSs for each equivalence class in L_j^-. For example, if we add configuration (iv) to

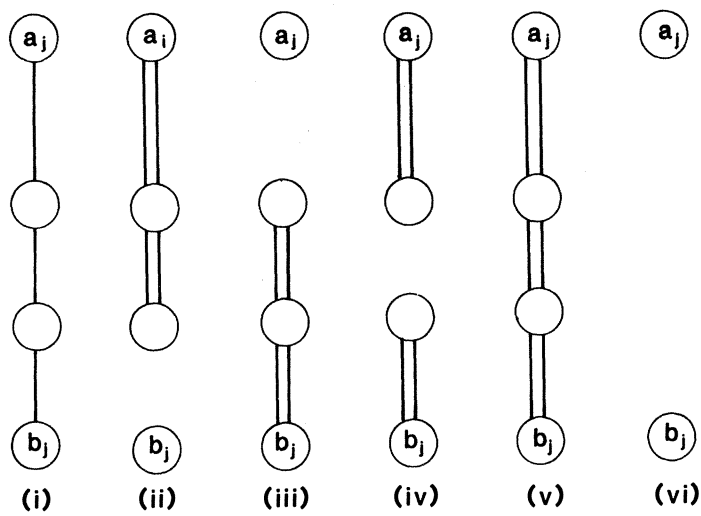


Figure 5. Possible arc configuration for any aisle j in an optimum tour subgraph.

a L_j^- PTS in equivalence class (0, E, 1C), we obtain a L_j^+ PTS in equivalence class (E, E, 2C). Obviously, we use only minimum length configurations.

If we have a minimum length L_j^- PTS for each equivalence class, we can construct a minimum length L_j^+ PTS for each equivalence class by simply selecting from Table I the least distance configuration that yields a L_j^+ PTS in each equivalence class.

To determine the minimum length L_{j+1}^- PTS for each equivalence class, given the minimum length L_j^+ PTS for each equivalence class, we must add one of the arc configurations indicated in Figure 6 corresponding to the crossover between aisle j and $j + 1$.

Table II indicates the equivalence class for L_{j+1}^- if the minimum length configurations in Figure 6 are added to the minimum length PTSs for

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

TABLE I

THE L_j^+ EQUIVALENCE CLASSES THAT RESULT FROM ADDING EACH OF THE CONFIGURATIONS IN FIGURE 5 TO THE PTSs IN THE L_j^- EQUIVALENCE CLASSES

L_j^- Equivalence Classes	Arc Configurations from Figure 5					
	(i)	(ii)	(iii)	(iv)	(v)	(vi)[a]
(U, U, 1C)	(E, E, 1C)	(U, U, 1C)	(U, U, 1C)	(U, U, 1C)	(U, U, 1C)	(U, U, 1C)
(E, 0, 1C)	(U, U, 1C)	(E, 0, 1C)	(E, E, 2C)	(E, E, 2C)	(E, E, 1C)	(E, 0, 1C)
(0, E, 1C)	(U, U, 1C)	(E, E, 2C)	(0, E, 1C)	(E, E, 2C)	(E, E, 1C)	(0, E, 1C)
(E, E, 1C)	(U, U, 1C)	(E, E, 1C)	(E, E, 1C)	(E, E, 1C)	(E, E, 1C)	(E, E, 1C)
(E, E, 2C)	(U, U, 1C)	(E, E, 2C)	(E, E, 2C)	(E, E, 2C)	(E, E, 1C)	(E, E, 2C)
(0, 0, 0C)[b]	(U, U, 1C)	(E, 0, 1C)	(0, E, 1C)	(E, E, 2C)	(E, E, 1C)	(0, 0, 0C)
(0, 0, 1C)[c]	—[d]	—	—	—	—	(0, 0, 1C)

[a] This is not a feasible configuration if there is any item to be picked in aisle j.
[b] This class can occur only if there are no items to be picked to the left of aisle j.
[c] This class is feasible only if there are no items to be picked to the right of aisle j.
[d] Could never be optimal.

each equivalence class in L_j^+. For example, adding configuration (iv) to a L_j^+ PTS in equivalence class (E, 0, 1C) creates a L_{j+1}^- PTS in equivalence class (E, E, 2C). The dashes indicate cases that could not lead to optimum tour subgraphs.

Hence, if we have a minimum length L_j^+ PTS for each equivalence

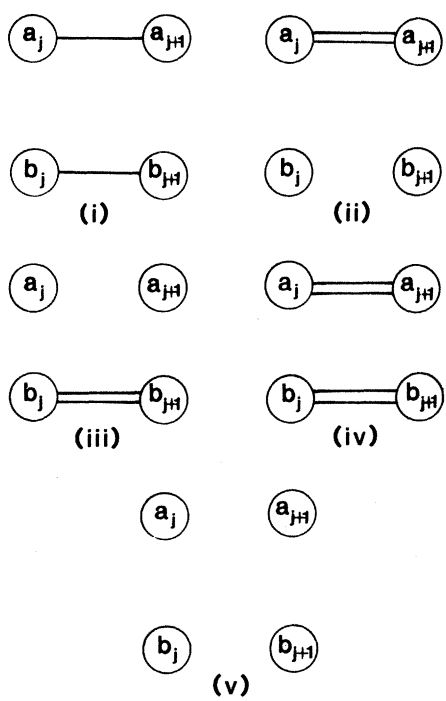


Figure 6. Possible arc configuration for crossover between aisle j and aisle $j + 1$ in an optimum tour subgraph.

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

class, we can construct a minimum length L_{j+1}^- PTS for each class by selecting from Table II the least distance configuration that yields a L_{j+1}^- PTS in each equivalence class.

Note that for $j = 1$, the six configurations in Figure 5 correspond exactly to the six equivalence classes in Corollary 3.1 for L_1^+ PTSs. Hence, a minimum length L_1^+ PTS for each equivalence class is very easy to determine. Using the procedure discussed above, we can then construct sequentially a minimum length L_j^- PTS for each equivalence class and then a L_j^+ PTS for each equivalence class for $j = 2, 3, \cdots, n$. From the minimum length L_n^+ PTS in equivalence classes (E, 0, 1C), (0, E, 1C), (E, E, 1C), and (0, 0, 1C), we will select the shortest as the minimum length order-picking tour subgraph.

TABLE II

THE L_{j+1}^- EQUIVALENCE CLASSES THAT RESULT FROM ADDING EACH OF THE CONFIGURATIONS IN FIGURE 6 TO THE PTSs IN THE L_j^+ EQUIVALENCE CLASSES

L_j^+ Equivalence Classes	Arc Configurations from Figure 6				
	(i)	(ii)	(iii)	(iv)	(v)
(U, U, 1C)	(U, U, 1C)	—[a]	—[a]	—[a]	—[a]
(E, 0, 1C)	—[a]	(E, 0, 1C)	—[b]	(E, E, 2C)	(0, 0, 1C)
(0, E, 1C)	—[a]	—[b]	(0, E, 1C)	(E, E, 2C)	(0, 0, 1C)
(E, E, 1C)	—[a]	(E, 0, 1C)	(0, E, 1C)	(E, E, 1C)	(0, 0, 1C)
(E, E, 2C)	—[a]	—[b]	—[b]	(E, E, 2C)	—[b]
(0, 0, 0C)	—[c]	—[c]	—[c]	—[c]	(0, 0, 0C)
(0, 0, 1C)	—[b]	—[b]	—[b]	—[b]	(0, 0, 1C)

[a] The degrees of a_j and b_j are odd.
[b] No completion can connect the graph.
[c] Would never be optimal.

3. NUMERICAL EXAMPLE

To illustrate the procedure, we again consider the example in Figures 1 and 2 with distances as indicated in Figure 7. Table III specifies the relevant information regarding the minimum length PTS for each equivalence class. The three numbers in each cell of the table are respectively: the length of the minimum length PTS for the equivalence class, the equivalence class of the predecessor PTS from which the current PTS was constructed, and the arc configuration (from Figure 5 for L_j^+ and Figure 6 for L_j^-) added to the predecessor PTS to obtain the current PTS. Equivalence classes are numbered as indicated at the left of the table.

There are never any L_1^- PTSs. The minimum length PTS in L_1^+ equivalence class (U, U, 1C) is obtained by simply adding configuration (i) from Figure 5 to a null graph. The length of the PTS is 15. The dash

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

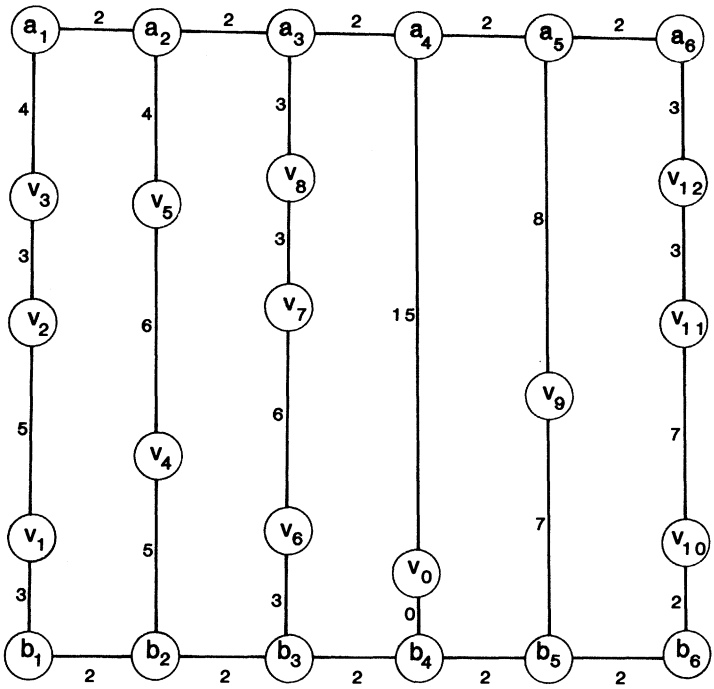


Figure 7. Example order-picking graph.

TABLE III

SOLUTION TO THE EXAMPLE PROBLEM IN FIGURE 7

Equivalence Class	Aisle 1		Aisle 2		Aisle 3	
	L_1^-	L_1^+	L_2^-	L_2^+	L_3^-	L_3^+
1. (U, U, 1C)	—[a]	15, –, i	19, 1, i	37, 1, iv	41, 1, i	53, 2, i
2. (E, 0, 1C)	—	24, –, ii	28, 2, ii	48, 2, ii	38, 4, ii	62, 2, ii
3. (0, E, 1C)	—	22, –, iii	26, 3, iii	48, 3, iii	38, 4, iii	62, 3, iii
4. (E, E, 1C)	—	30, –, v	38, 4, iv	34, 1, i	42, 4, iv	56, 1, i
5. (E, E, 2C)	—	20, –, iv	28, 5, iv	44, 3, 4	52, 5, iv	56, 2, iv
6. (0, 0, 0C)	—	—	—	—	—	—
7. (0, 0, 1C)	—	—	22, 3, v	—	34, 4, v	—

	Aisle 4		Aisle 5		Aisle 6	
	L_4^-	L_4^+	L_5^-	L_5^+	L_6^-	L_6^+
1. (U, U, 1C)	57, 1, i	57, 1, iii	61, 1, i	75, 1, iii	79, 1, i	95, 1, iv
2. (E, 0, 1C)	60, 4, ii	90, 2, ii	68, 4, ii	84, 2, ii	80, 4, ii	106, 2, ii
3. (0, E, 1C)	60, 4, iii	60, 3, iii	64, 3, iii	78. 3, iii	80, 4, iii	104, 3, iii
4. (E, E, 1C)	64, 4, iv	64, 4, iii	72, 4, iv	76, 1, i	84, 4, iv	94, 1, i
5. (E, E, 2C)	64, 5, iv	60, 2, iii	68, 3, iv	80, 3, ii	86, 3, iv	96, 2, iv
6. (0, 0, 0C)	—	—	—	—	—	—
7. (0, 0, 1C)	56, 4, v	—	60, 3, v	—	76, 4, v	—

[a] There is no PTS for the equivalence class.

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

(—) indicates that there was no predecessor PTS. The other minimum length PTSs in the L_1^+ column are also determined by adding the indicated configuration from Figure 5 to a null graph (e.g., (E, E, 1C) is obtained by adding configuration (v) to a null graph).

To obtain the L_2^- column we use Table II. For example, we see from Table II that there are two ways to obtain an L_2^- PTS in equivalence class (E, 0, 1C). We could add arc configuration (ii) from Figure 6 to the PTS in either the (E, 0, 1C) or the (E, E, 1C) equivalence class in L_1^+. From Table III we see that the PTS in (E, 0, 1C) has length 24 and the PTS in (E, E, 1C) has length 30. Hence, the minimum length PTS for the (E, 0, 1C) equivalence class in L_2^- is obtained by adding arc configuration (ii) from Figure 6 to the minimum length PTS in (E, 0, 1C) of L_1^+. The length of the resulting PTS is 28. Similarly, we obtain the minimum length PTSs in the L_j^+ equivalence classes for $j = 2, 3, \cdots, 6$ sequentially from the L_j^- equivalence classes using Table I by adding the arc configurations from Figure 5.

The optimum order-picking tour subgraph is the shortest of the PTSs in (E, 0, 1C), (0, E, 1C), (E, E, 1C), or (0, 0, 1C) of L_6^+. From the L_6^+ column of Table III, we find that the optimum order-picking tour subgraph is in (E, E, 1C) and has length 94. The sequence of arc configurations added can then be traced back through Table III as indicated by the underlined elements. To construct the order-picking tour graph, simply add the arc configurations from Figure 5 and 6 indicated by the underlined elements in Table III for each aisle and crossover. Figure 8 shows the resulting minimum length tour subgraph.

4. TOUR CONSTRUCTION

We have developed a procedure for finding a minimum length tour subgraph T from the order-picking graph. There remains the question of how to construct a tour from T. The following is a very simple and efficient procedure for constructing an optimum order-picking tour from T.

Step 1. Begin the tour by letting v_0 be the first vertex visited.
Step 2. Let v^* be the vertex currently being visited.
Step 3. If there is a pair of unused parallel arcs in T incident to v^*, use one of them to get to the next vertex. Go to Step 2.
Step 4. If there are any unused single arcs in T (i.e., not one of a pair of parallel arcs), use one of them to get to the next vertex. Go to Step 2.
Step 5. If there is a pair of parallel arcs in T with one still unused, use it to get to the next vertex. Go to Step 2.
Step 6. Stop. The order-picking tour is complete.

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

We first note that this procedure constructs a cycle in the tour subgraph T which starts at v_0. Hence, it must also stop at v_0 since each vertex has an even number of edges incident to it. Every edge incident to v_0 must be included in the cycle or else the procedure would not have stopped. The only question left to resolve is whether or not the cycle includes every edge in T. To establish that this is so, we will use the following results.

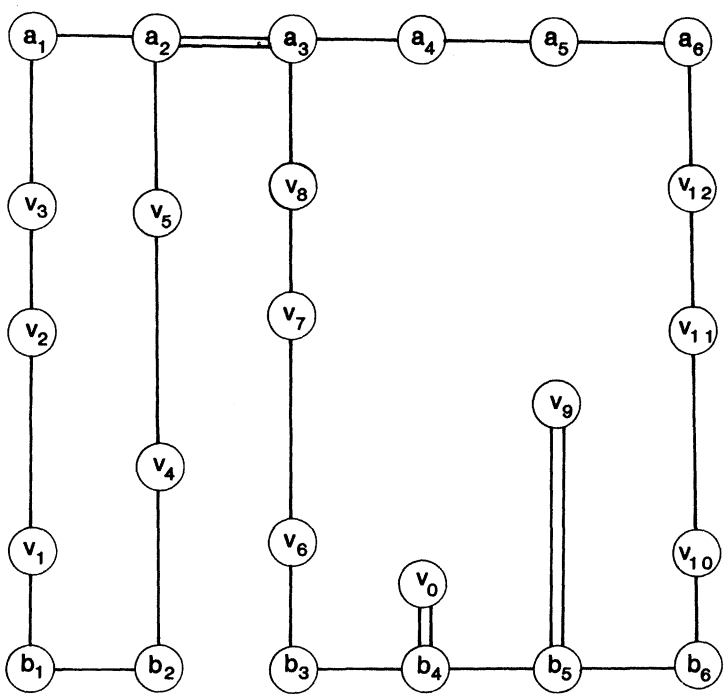


Figure 8. Optimum order-picking tour subgraph for the problem in Figure 7 and Table III.

LEMMA 1. *If a_1 and a_2 are a pair or parallel arcs in a minimum length tour subgraph T, then no two arc disjoint cycles in T can have a_1 in one and a_2 in the other.*

Proof. Suppose that there are two such cycles. If we delete a_1 and a_2 from T, the degree of each vertex in T is still even and T is still connected. This contradicts the assumption that T is a minimum length tour subgraph.

LEMMA 2. *The tour construction procedure cannot include only one of the pair of parallel arcs a_1 and a_2 in the cycle.*

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

Proof. Assume that only one of the pair, say a_1 were included in the cycle. We can then construct a second cycle which includes a_2 by starting with the vertex at one end of a_2 and adding incident edges and vertices not in the first cycle until we reach the vertex on the other end of a_2. From Lemma 1, this contradicts the assumption that T is minimum length tour subgraph.

THEOREM 4. *The tour construction procedure includes every arc of T in the cycle.*

Proof. Assume that some arc of T is not included in the cycle. Since T is connected, there must be an arc, say a, not included in the cycle which has at least one of its vertices, say v' in the cycle. The degree of v' in T is even and the cycle uses an even number of arcs incident to v'. Hence, there are an even number of arcs incident to v' that are not in the cycle. From Lemma 2 any pair of parallel arcs incident to v' are either both in the cycle or neither is in the cycle. Therefore, if a' is one of a parallel pair of arcs, neither are in the cycle.

Now consider the last time the cycle visited v'. It could not have left v' via one of a pair of parallel arcs since this would violate Lemma 2 if neither were in the cycle already, and a' would have taken precedence if one of the pair was already in the cycle. Hence, it must have left via a single arc. This implies that a' is also a single arc since it would have taken precedence if it were one of an unused pair.

A vertex in T has at most two single arcs incident to it. Since v' is incident to one single arc in the cycle and one single arc not in the cycle and since the number of unused arcs incident to v' is even, there must be a pair of arcs incident to v' with one in the cycle and the other not in the cycles. From Lemma 2 this is a contradiction.

5. EXTENSIONS AND CONCLUSIONS

The method presented here provides a very efficient procedure to solve the order-picking problem for the aisle configuration given in Figure 1. A few variations in this configuration can be handled without much increase in computational effort.

The loading dock can be along any aisle without altering the formulation. If the dock is on a crossover, say between b_j and b_{j+1}, then when considering L_{j+1}^- PTSs, we could force the v_0 vertex into every PTS. Another alternative would be to insert an "artificial aisle" containing v_0, but with length sufficiently long to keep it from being transversed.

If there are items stored along crossovers, the procedure requires only a minor change in going from the L_j^+ equivalence class to the L_{j+1}^- equivalence classes. The configurations in Figure 6 must be expanded to include the possibility of only partially traversing the crossover. Also the

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions

partial tour subgraph must obviously include the v_i along the crossover. This extension is straightforward.

The aisles do not have to be the same length and the crossovers do not have to be the same length. Hence, other configurations can be handled as long as they retain the same basic structure of crossovers only at the ends of the aisles.

The basic ideas can also be extended to the case where crossovers are allowed within the aisles as well as at the ends of aisles. However this dramatically increases the number of equivalence classes that must be considered. For the case where crossovers within aisles were not allowed, we only had to be concerned with combinations of degrees for the two vertices corresponding to the ends of the aisle. If we allow p crossovers within an aisle, then we have to consider degree combinations for $p + 2$ vertices. If we allow crossovers within aisles, the number of connectivity classes also increases. In addition, to specify the equivalence class if there are two connected components, we need to know which of the intersection nodes are in the same component. With only a few crossovers, the procedure would still require a reasonable amount of computational effort. However, the procedure does not seem practical for more than two or three crossovers within each aisle.

ACKNOWLEDGMENT

This research was sponsored in part by the Office of Naval Research under the Production and Distribution Research Center at the Georgia Institute of Technology.

REFERENCES

CHRISTOFIDES, N. 1975. *Graph Theory: An Algorithmic Approach.* Academic Press, London.

GILMORE, P. C., AND R. E. GOMORY. 1964. Sequencing a One State-Variable Machine: A Solvable Case of the Traveling Salesman Problem. *Opns. Res.* **12,** 655–679.

LAWLER, E. L. 1971. A Solvable Case of the Traveling Salesman Problem. *Math. Program.* **1,** 267–269.

This content downloaded from 128.29.43.1 on Wed, 24 Sep 2014 09:31:29 AM
All use subject to JSTOR Terms and Conditions